Leitura — Responsividade, acessibilidade, navegação e testes (Sprint 1, parte 2)

Guia de apoio para a segunda parte da Sprint 1 (vídeo e aulas de 05 e 07/10). A tela de tarefas de 14/09 ganha uma segunda tela (detalhe), navegação entre as duas, um layout que muda com a largura da janela, ajustes de acessibilidade e testes. São exatamente os critérios da rubrica da Sprint 1: navegação com rotas tipadas e deep link, layout adaptado a pelo menos duas larguras, acessibilidade e pelo menos cinco testes de interface.

Exemplo de referência (Android + Desktop): `exemplos/tarefas-compose/` do repositório `sistemas-moveis-2026-2`, passos 6 a 10 do `PASSOS.md` (os passos 4 e 5, formulário e tema, estão nos capítulos 8 e 9 da leitura de 14/09). Versões: Kotlin 2.4.10, Compose Multiplatform 1.12.0, Navigation Compose multiplataforma 2.9.2, Material 3 Adaptive 1.2.0, Android Gradle Plugin 9.1.0, compileSdk e targetSdk 37, minSdk 24.

Os comportamentos descritos foram conferidos em execução:

- testes de interface do Compose no alvo Desktop (8 testes do exemplo, mais testes descartáveis para medir larguras e imprimir a árvore de semântica);
- o APK no emulador Android (AVD Pixel 8, Android 17, API 37): deep link por `adb`, botão Voltar do sistema e rotação da tela.

Onde rodar. O projeto usa AGP 9.1, que exige o Android Studio Panda 2 (2025.3.2) ou mais novo. Há três caminhos, do mais leve ao mais completo:

Caminho	Precisa de	Serve para
Alvo desktop, com Hot Reload: `./gradlew :composeApp:hotRunDesktop --auto`	só o JDK; sem Android Studio nem emulador	construir as telas e rodar os testes
Codespaces do repositório (Code → Codespaces)	um navegador	Android Studio com `@Preview`, Hot Reload e geração do APK, sem instalar nada
Android Studio na própria máquina, com emulador	Panda 2 ou mais novo e memória para o emulador	rodar no Android, deep link, TalkBack

No laboratório, cujo Android Studio é anterior ao Panda 2, o caminho é o Codespaces. Ele serve também para quem tem um computador em que o Android Studio ou o emulador ficam pesados: o trabalho roda na nuvem, e a máquina só exibe o navegador. Para ver o app no Android sem emulador, gere o APK no Codespace e instale num celular. O passo a passo está na seção *Como abrir e rodar* de `exemplos/tarefas-compose/PASSOS.md`, e a seção 7.3 responde às dúvidas mais comuns.

Capítulos:

1. O que muda no exemplo
2. Responsividade e adaptatividade
3. Acessibilidade
4. Navegação com Navigation Compose
5. Testes: `kotlin.test` e teste de interface
6. Oficina: testes para o projeto
7. Exercícios e dúvidas frequentes

---

1. O que muda no exemplo

1.1 Arquivos

Em 14/09, toda a interface cabia em `App.kt`. Com duas telas, o código foi dividido:

Arquivo	O que tem
`Tarefa.kt`	o modelo e as regras como funções puras (`tituloValido` , `comNova` , `alternando`)
`Telas.kt`	`TelaLista` , `TelaDetalhe` , `CartaoTarefa` , `FormularioTarefa` e os previews
`App.kt`	`App()` , as rotas, o estado e a escolha entre uma tela ou duas (`Conteudo`)
`commonTest/`	`RegrasTest` (`kotlin.test`) e `TelasTest` (teste de interface)

1.2 Regras fora do Compose

A lógica que estava dentro dos lambdas da tela virou funções puras:

```
fun tituloValido(texto: String): Boolean = texto.isNotBlank()


fun List<Tarefa>.comNova(id: Int, titulo: String): List<Tarefa> = this + Tarefa(id,
        titulo.trim())


fun List<Tarefa>.alternando(id: Int): List<Tarefa> =
    map { if (it.id == id) it.copy(feita = !it.feita) else it }
```

`comNova` e `alternando` são funções de extensão: declaradas fora da classe `List` , mas chamadas como se fossem dela, `tarefas.alternando(1)` . Continuam imutáveis: devolvem uma lista nova, e a tela reatribui o estado. A vantagem é que dá para testá-las sem montar tela nenhuma (seção 5.2).

1.3 O estado subiu mais um nível

Com duas telas, a lista não pode morar em nenhuma delas: as duas precisam enxergá-la. Ela fica em `Conteudo`, acima da navegação, e desce como parâmetro:

```
@Composable
fun Conteudo(
    largo: Boolean,
    modifier: Modifier = Modifier,
    navController: NavHostController = rememberNavController(),
) {
    var tarefas by remember { mutableStateOf(tarefasIniciais) }
    var texto by rememberSaveable { mutableStateOf("") }
    var proximoId by remember { mutableStateOf(3) }
    var selecionada by rememberSaveable { mutableStateOf<Int?>(null) }
    // ...
}
```

É o mesmo estado elevado de 14/09, um nível acima. Na Sprint 2, esse estado vai para um `ViewModel`, que sobrevive à recriação da tela e sai da árvore de composição.

1.4 O import do `@Preview`

Há duas anotações `@Preview` com o mesmo nome, e só uma compila com a dependência do exemplo (`org.jetbrains.compose.ui:ui-tooling-preview`):

```
import androidx.compose.ui.tooling.preview.Preview          // esta
import org.jetbrains.compose.ui.tooling.preview.Preview     // descontinuada, não compila
```

A leitura de 14/09 (seção 2.6) explica a diferença entre as duas.

---

2. Responsividade e adaptatividade

2.1 Os dois termos

- Responsivo: o mesmo layout se estica ou encolhe para caber (uma lista que ocupa a largura toda).
- Adaptativo: o layout muda de forma conforme o espaço (uma coluna no celular, duas no tablet).

A rubrica pede layout adaptado a pelo menos duas larguras, sem overflow (conteúdo cortado ou vazando da tela).

2.2 Classes de tamanho de janela

Em vez de perguntar "é celular ou tablet?", o Android e o Compose Multiplatform perguntam "quanto espaço esta janela tem?". A resposta vem em classes com limites fixos, em dp:

Classe de largura	Largura	Exemplos
Compacta	menos de 600 dp	celular em pé
Média	600 a 839 dp	celular deitado, tablet pequeno em pé, dobrável aberto
Expandida	840 dp ou mais	tablet deitado, janela de desktop

Por que janela e não aparelho: no Android há tela dividida, janelas redimensionáveis e dobráveis; no desktop, a janela tem o tamanho que o usuário quiser. O mesmo aparelho passa por várias classes.

Por que não orientação (retrato × paisagem): um tablet em pé tem mais largura que um celular deitado. A largura é o que decide se cabem duas colunas.

📖 Ref. Android — Use window size classes: https://developer.android.com/develop/ui/compose/layouts/adaptive/use-window-size-classes

2.3 No código

Dependência: `org.jetbrains.compose.material3.adaptive:adaptive:1.2.0`.

```
@Composable
fun App() {
    // Largura da janela: compacta (celular em pé) ou média/expandida (tablet, desktop).
    val largura = currentWindowAdaptiveInfo().windowSizeClass
    val largo =
        largura.isWidthAtLeastBreakpoint(WindowSizeClass.WIDTH_DP_MEDIUM_LOWER_BOUND)

    // O tema segue o sistema. `MaterialTheme { }` sem `colorScheme` é sempre claro.
    val cores = if (isSystemInDarkTheme()) darkColorScheme() else lightColorScheme()

    MaterialTheme(colorScheme = cores) {
        Surface(Modifier.fillMaxSize()) {
            Conteudo(largo = largo, modifier = Modifier.safeDrawingPadding())
        }
    }
}
```

- `currentWindowAdaptiveInfo()` lê a janela atual e se recompõe quando ela muda de tamanho.

- `isWidthAtLeastBreakpoint(WIDTH_DP_MEDIUM_LOWER_BOUND)` pergunta "a largura é pelo menos média (600 dp)?". Tutoriais mais antigos comparam `windowWidthSizeClass` com `WindowWidthSizeClass.COMPACT`; o exemplo usa a forma por limite, a da documentação atual.
- `App()` decide e passa um `Boolean` para `Conteudo`. Assim os testes escolhem o layout sem mexer no tamanho da janela (seção 5.4).

Conferido no desktop, com `App()` numa janela de teste: com 599 px de largura, uma tela por vez; com 600, os dois painéis. No emulador (Pixel), em pé é compacta; ao girar para paisagem, a mesma tela passou a mostrar lista e detalhe lado a lado, sem reiniciar o app.

2.4 Lista e detalhe

```
if (largo) {
    // Janela larga: lista e detalhe lado a lado, sem navegar.
    Row(modifier.fillMaxSize()) {
        TelaLista(
            tarefas, texto, { texto = it }, adicionar, alternar,
            onAbrir = { selecionada = it },
            modifier = Modifier.width(360.dp),
        )
        VerticalDivider()
        TelaDetalhe(
            tarefa = tarefas.find { it.id == selecionada },
            onAlternar = { selecionada?.let(alternar) },
            onVoltar = null,
            modifier = Modifier.fillMaxHeight(),
        )
    }
} else {
    // Janela compacta: uma tela por vez, com pilha de retorno (capítulo 4).
    NavHost(/* ... */)
}
```

- As telas são as mesmas nos dois modos. Muda quem as organiza e o que acontece em "Detalhes": no modo largo, selecionar; no compacto, navegar.
- `TelaDetalhe` recebe `onVoltar: (() -> Unit)?`. No painel lateral não há para onde voltar, então o botão some (`null`).
- A lista tem largura fixa (360 dp) e o detalhe ocupa o resto. Em janelas muito largas, o detalhe fica espaçoso; limitar a largura máxima do conteúdo é um bom exercício.

Para layouts mais elaborados, o Material 3 Adaptive tem componentes prontos (`ListDetailPaneScaffold`, `NavigationSuiteScaffold`). A versão manual acima mostra a ideia com menos peças.

📖 Ref. Compose Multiplatform — Adaptive layouts: https://kotlinlang.org/docs/multiplatform/compose-adaptive-layouts.html

2.5 Tema, `Surface` e áreas seguras

Três peças na raiz do `App()`, que entram no Passo 5 do exemplo:

- `MaterialTheme(colorScheme = ...)` com `isSystemInDarkTheme()`: o tema acompanha o sistema. `MaterialTheme { }` sem `colorScheme` é sempre claro.
- `Surface(Modifier.fillMaxSize())` pinta o fundo com a cor do tema e dá ao texto a cor de conteúdo correspondente. Sem ela, no modo escuro os componentes escurecem, e o fundo e o texto fora de componentes continuam claros.
- `Modifier.safeDrawingPadding()` afasta o conteúdo da barra de status, da barra de navegação e do recorte da câmera. Com targetSdk 35 ou mais, o Android desenha o app de ponta a ponta (edge-to-edge), e sem esse afastamento o título fica embaixo da barra de status.

No Android, o `MainActivity` chama `enableEdgeToEdge()` antes do `setContent`: ele escolhe a cor dos ícones da barra de status pelo tema, escuros no modo claro e claros no modo escuro.

Conferido no emulador, nos dois modos, com `adb shell cmd uimode night yes` e `night no`.

📖 Ref. Android — Window insets in Compose: https://developer.android.com/develop/ui/compose/system/insets

> Erro comum: decidir o layout pela orientação (`LocalConfiguration.current.orientation`) ou por "é tablet?". Um celular deitado em tela dividida é estreito; um tablet em pé é largo. Use a largura da janela.

> Erro comum: testar só no celular em pé. A rubrica pede duas larguras. No desktop, basta redimensionar a janela; no emulador, girar a tela.

---

3. Acessibilidade

3.1 Quem usa e como

Acessibilidade é o app funcionar para quem não enxerga a tela, enxerga pouco, não usa as mãos com precisão ou usa teclado. No Android, o leitor de tela é o TalkBack; ele não vê pixels, vê uma árvore de semântica que o Compose monta a partir dos componentes: cada nó tem papel (botão, caixa de seleção), texto, estado e ações.

A rubrica pede: descrições de conteúdo nos elementos interativos, contraste verificado, alvos de toque de pelo menos 48 dp e navegação por leitor de tela testada.

📖 Ref. Android — Accessibility in Compose: https://developer.android.com/develop/ui/compose/accessibility

📖 Ref. Android — Semantics: https://developer.android.com/develop/ui/compose/accessibility/semantics

3.2 A árvore de semântica, antes e depois

O cartão de 14/09 tinha uma `Checkbox` com `onCheckedChange` e um `Text` ao lado. A árvore dessa versão, impressa num teste com `onRoot().printToString()` (trecho):

```
|-Node #3 at (l=12.0, t=12.0, r=36.0, b=36.0)px
| Role = 'Checkbox'
| ToggleableState = 'Off'
| Actions = [OnClick, OnFillData, RequestFocus]
|-Node #4 at (l=48.0, t=0.0, r=166.0, b=16.0)px
  Text = '[Estudar Compose]'
```

Dois nós separados: a caixa sem texto e o texto sem ação. O leitor de tela anuncia uma caixa de seleção sem dizer de quê, e tocar no texto não faz nada.

A versão do Passo 6:

```
Row(
    Modifier
        .weight(1f)
        .toggleable(value = tarefa.feita, role = Role.Checkbox, onValueChange = {
        onAlternar() })
        .padding(12.dp),
    verticalAlignment = Alignment.CenterVertically,
) {
    Checkbox(checked = tarefa.feita, onCheckedChange = null) // o clique é da linha
    Spacer(Modifier.width(8.dp))
    Text(tarefa.titulo)
}
```

E a árvore:

```
    |–Node #9 at (l=0.0, t=0.0, r=939.0, b=48.0)px
    | Role = 'Checkbox'
    | Text = '[Estudar Compose]'
    | ToggleableState = 'Off'
    | Actions = [..., OnClick, ...]
    | MergeDescendants = 'true'
```

- `toggleable` torna a linha inteira alternável e, com `role = Role.Checkbox`, diz ao leitor de tela o que ela é.
- `MergeDescendants = 'true'`: os filhos (a caixa e o texto) viram um nó só. O leitor de tela lê o conjunto de uma vez.
- `onCheckedChange = null` tira da `Checkbox` o clique próprio. Sem isso, haveria dois elementos clicáveis sobrepostos, com dois anúncios.
- A área de toque é a linha toda (48 dp de altura no teste), não só o quadradinho.

O botão "Detalhes" fica fora da área alternável, como nó separado: é outra ação.

📖 Ref. Android — Merging and clearing semantics:
https://developer.android.com/develop/ui/compose/accessibility/merging-clearing

3.3 Títulos

```
Text(
    "Minhas tarefas",
    style = MaterialTheme.typography.headlineSmall,
    modifier = Modifier.semantics { heading() },
)
```

O estilo grande só muda a aparência. `heading()` marca o texto como título na árvore, e o leitor de tela permite pular de título em título. O teste da seção 5.3 confere que a tela tem um título.

3.4 Descrições de conteúdo

Texto visível já é lido. O que não tem texto precisa de descrição: ícones, imagens, botões só com ícone.

```
IconButton(onClick = onApagar) {
    Icon(Icons.Default.Delete, contentDescription = "Apagar tarefa")
}

// Imagem decorativa: nada a anunciar.
Image(painter = fundo, contentDescription = null)
```

- A descrição diz o que o elemento faz ou mostra, não como ele é ("Apagar tarefa", não "ícone de lixeira").
- `contentDescription = null` é uma decisão: o elemento é decorativo e deve ser ignorado.
- Não repita o papel: o leitor de tela já anuncia "botão"; "botão apagar" vira "botão apagar, botão".

O exemplo usa botões com texto ("Detalhes", "Voltar"), que não precisam de descrição.

3.5 Alvos de toque e contraste

- Alvos de toque: pelo menos 48 × 48 dp. Os componentes Material 3 (`Checkbox` , `IconButton` , `Button`) já garantem esse mínimo. O risco está nos elementos feitos à mão com `clickable` num ícone pequeno: use `Modifier.minimumInteractiveComponentSize()` ou aumente a área com `padding` antes do `clickable` .
- Contraste: usar as cores do tema pelos papéis (`onSurface` sobre `surface` , `onPrimary` sobre `primary`) garante pares pensados para contraste. Cores fixas à mão (um cinza claro sobre branco) são o erro mais comum. Para verificar, o app Accessibility Scanner, no Android, aponta contraste baixo e alvos pequenos na tela aberta.
- Texto em `sp` (o padrão da tipografia do tema) acompanha o tamanho de fonte escolhido pelo usuário nas configurações.

📖 Ref. Android — Key steps to improve Compose accessibility: https://developer.android.com/develop/ui/compose/accessibility/key-steps

📖 Ref. WCAG 2.2 — Target Size (Minimum): https://www.w3.org/WAI/WCAG22/Understanding/target-size-minimum.html

3.6 Testar com o leitor de tela

A rubrica pede navegação por leitor de tela testada. No emulador ou no celular:

1. Configurações › Acessibilidade › TalkBack › ativar. Algumas imagens de emulador não trazem o TalkBack; ele vem no app Android Accessibility Suite.
2. Deslizar para a direita passa ao próximo elemento; toque duplo ativa.
3. Percorrer a tela inteira e conferir: cada elemento é anunciado com um nome que faz sentido, na ordem de leitura, e toda ação é alcançável.

No desktop, o Compose também expõe a árvore de semântica para os leitores de tela do sistema (VoiceOver no macOS, por exemplo).

📖 Ref. Compose Multiplatform — Accessibility: https://kotlinlang.org/docs/multiplatform/compose-accessibility.html

> Erro comum: `clickable` só no `Text` e a `Checkbox` com o próprio clique. Dois alvos, dois anúncios, e o usuário precisa acertar o certo. Um `toggleable` na linha resolve.

> Erro comum: `contentDescription` num elemento que já tem texto. O leitor de tela lê a descrição e o texto, ou só a descrição, e o que está escrito na tela deixa de bater com o que é anunciado.

---

4. Navegação com Navigation Compose

4.1 Por que um grafo de navegação

Dá para trocar de tela com um `if` e uma variável de estado. A rubrica classifica isso como insuficiente, e por bons motivos: sem grafo não há pilha de retorno (o Voltar do sistema fecha o app), nem argumentos tipados, nem deep links. O Navigation Compose resolve os três.

Dependências (multiplataforma, da JetBrains) e o plugin de serialização:

```
navigation = "2.9.2"
navigation-compose = { module = "org.jetbrains.androidx.navigation:navigation-compose",
        version.ref = "navigation" }
kotlinSerialization = { id = "org.jetbrains.kotlin.plugin.serialization", version.ref =
        "kotlin" }
```

📖 Ref. Compose Multiplatform — Navigation: https://kotlinlang.org/docs/multiplatform/compose-navigation.html

4.2 Rotas tipadas

Cada destino é um tipo `@Serializable`. Argumentos são propriedades:

```
@Serializable
object Lista


@Serializable
data class Detalhe(val id: Int)
```

- `Lista` não tem argumentos: um `object` basta.
- `Detalhe(val id: Int)` carrega o id da tarefa. Navegar é construir o objeto: `navController.navigate(Detalhe(2))`.
- O compilador confere o tipo do argumento. Nas versões antigas, rotas eram textos como `"detalhe/{id}"`, e um erro de digitação só aparecia ao rodar.

📖 Ref. Android — Type safety in Kotlin DSL and Navigation Compose: https://developer.android.com/guide/navigation/design/type-safety

4.3 O grafo

```
NavHost(navController, startDestination = Lista, modifier = modifier) {
    composable<Lista> {
        TelaLista(
            tarefas, texto, { texto = it }, adicionar, alternar,
            onAbrir = { id -> navController.navigate(Detalhe(id)) },
        )
    }
    composable<Detalhe>(
        // tarefas://tarefa/2 abre direto o detalhe da tarefa 2 (Android).
        deepLinks = listOf(navDeepLink<Detalhe>(basePath = "tarefas://tarefa")),
    ) { entrada ->
        val rota = entrada.toRoute<Detalhe>()
        TelaDetalhe(
            tarefa = tarefas.find { it.id == rota.id },
            onAlternar = { alternar(rota.id) },
            onVoltar = { navController.popBackStack() },
        )
    }
}
```

- `NavHost` mostra o destino do topo da pilha. `startDestination = Lista` é a base.
- `composable<Detalhe>` registra o destino; `entrada.toRoute<Detalhe>()` reconstrói o objeto com o argumento.
- As telas não conhecem o `navController`. Recebem lambdas (`onAbrir`, `onVoltar`), e quem navega é o grafo. Isso mantém as telas testáveis e com preview.
- O argumento é só o id. A tarefa em si vem do estado (`tarefas.find`). Passar o objeto inteiro na rota duplicaria o dado, e a cópia ficaria velha quando a lista mudasse.

4.4 Pilha de retorno

- `navigate(Detalhe(id))` empilha o detalhe sobre a lista.
- `popBackStack()` desempilha. O botão "Voltar" do exemplo faz isso.
- No Android, o Voltar do sistema (gesto ou botão) também desempilha: conferido com `adb shell input keyevent KEYCODE_BACK`, que voltou do detalhe para a lista. Com a lista sozinha na pilha, o Voltar fecha o app.
- No desktop não há Voltar do sistema; por isso o botão na tela.

O estado das tarefas mora acima do `NavHost`: marcar uma tarefa como feita no detalhe e voltar mostra a linha marcada na lista (é o que o teste `compactaNavegaParaODetalheEVolta` confere).

📖 Ref. Android — Back stack: https://developer.android.com/guide/navigation/backstack

4.5 Deep links

Deep link é um endereço que abre o app direto num destino, vindo de um link, de uma notificação ou de outro app. No exemplo, `tarefas://tarefa/2` abre o detalhe da tarefa 2.

Duas partes:

1. No grafo, o destino declara o padrão: `navDeepLink<Detalhe>(basePath = "tarefas://tarefa")`. Os argumentos da rota viram segmentos do caminho: `tarefas://tarefa/{id}`.
2. No Android, o `AndroidManifest.xml` avisa ao sistema que a Activity atende esse endereço:

```
<intent-filter>
    <action android:name="android.intent.action.VIEW" />
    <category android:name="android.intent.category.DEFAULT" />
    <category android:name="android.intent.category.BROWSABLE" />
    <data android:scheme="tarefas" android:host="tarefa" />
</intent-filter>
```

Para testar com o app instalado no emulador:

```
adb shell am start -W -a android.intent.action.VIEW -d "tarefas://tarefa/2"
        br.ufrn.exemplo.tarefas
```

Conferido: o app abriu direto no detalhe de "Entender estado elevado". O Voltar levou para a lista, e não para fora do app: ao abrir por deep link, a navegação monta a pilha a partir do destino inicial.

A rubrica pede pelo menos um deep link funcional demonstrado. O comando acima serve como demonstração; um link numa página web ou numa notificação também.

Dá para exercitar o mesmo caminho **sem emulador**, num teste de interface no alvo desktop: `NavUri` é multiplataforma no Navigation 2.9.2, e `navController.navigate(NavUri(...))` resolve o endereço pelo mesmo `navDeepLink` do grafo.

```
lateinit var navegacao: NavHostController
setContent {
    navegacao = rememberNavController()
    Conteudo(largo = false, navController = navegacao)
}
runOnIdle { navegacao.navigate(NavUri("tarefas://tarefa/2")) }
waitForIdle()
onNodeWithText("Situação: pendente").assertIsDisplayed()
```

Isso não substitui a demonstração no aparelho — o `intent-filter` do manifesto só é exercitado lá —, mas põe o deep link sob teste automático, e não só num vídeo.

📖 Ref. Compose Multiplatform — Deep links: https://kotlinlang.org/docs/multiplatform/compose-navigation-deep-links.html

📖 Ref. Android — Create a deep link for a destination: https://developer.android.com/guide/navigation/navigation-deep-link

4.6 Aninhamento

A rubrica menciona aninhamento "onde faz sentido". Um grafo aninhado agrupa destinos de um fluxo (por exemplo, as telas de cadastro) sob uma rota própria, com destino inicial próprio. Faz sentido quando o projeto tem um fluxo de várias telas que entra e sai como unidade. Com duas telas, como no exemplo, não faz.

📖 Ref. Compose Multiplatform — Navigation and routing: https://kotlinlang.org/docs/multiplatform/compose-navigation-routing.html

> Erro comum: passar o `navController` para dentro das telas. A tela fica presa à navegação e perde o preview e o teste isolado. Passe lambdas.

> Erro comum: esquecer o plugin `kotlin("plugin.serialization")`. As rotas são `@Serializable` e, sem o plugin, a anotação não gera nada.

> Erro comum: deep link declarado só no grafo, sem o `intent-filter`. O Android não sabe que o app atende aquele endereço, e o comando `adb` não encontra quem abra o link.

---

5. Testes: `kotlin.test` e teste de interface

5.1 Onde ficam e como rodam

Os testes ficam em `composeApp/src/commonTest`: valem para todos os alvos. Os de interface rodam no alvo desktop, que não precisa de emulador:

```
./gradlew :composeApp:desktopTest
```

Dependências:

```
commonTest.dependencies {
    implementation(kotlin("test"))
    implementation(libs.compose.ui.test) // org.jetbrains.compose.ui:ui-test
}
val desktopTest by getting {
    dependencies { implementation(compose.desktop.currentOs) }
}
```

Os 8 testes do exemplo (3 de regras, 5 de interface) rodaram em cerca de 1,6 segundo, com o Gradle já aquecido.

📖 Ref. Compose Multiplatform — Testing Compose UI:
https://kotlinlang.org/docs/multiplatform/compose-test.html

5.2 `kotlin.test` : regras puras

```
class RegrasTest {

    @Test
    fun tituloEmBrancoNaoEValido() {
        assertFalse(tituloValido(""))
        assertFalse(tituloValido("   "))
        assertTrue(tituloValido("Ler"))
    }

    @Test
    fun novaTarefaEntraNoFimSemEspacosNasPontas() {
        val lista = tarefasIniciais.comNova(3, "  Revisar  ")
        assertEquals(Tarefa(3, "Revisar"), lista.last())
        assertEquals(3, lista.size)
    }

    @Test
    fun alternarMudaSoATarefaDoId() {
        val lista = tarefasIniciais.alternando(1)
        assertTrue(lista.first { it.id == 1 }.feita)
        assertFalse(lista.first { it.id == 2 }.feita)
    }
}
```

`kotlin.test` é a biblioteca de testes multiplataforma do Kotlin: `@Test`, `assertEquals`, `assertTrue`. Na JVM ela usa o JUnit por baixo; em outros alvos, o executor de cada plataforma. Estes testes rodam em milissegundos (0,018 s os três).

📖 Ref. kotlin.test: https://kotlinlang.org/api/core/kotlin-test/

5.3 Teste de interface

```
@OptIn(ExperimentalTestApi::class)
class TelasTest {

    @Test
    fun adicionaTarefaPeloFormulario() = runComposeUiTest {
        setContent { Conteudo(largo = false) }
        onNodeWithText("Adicionar").assertIsNotEnabled()
        onNodeWithText("Nova tarefa").performTextInput("Escrever testes")
        onNodeWithText("Adicionar").performClick()
        onNodeWithText("Escrever testes").assertIsDisplayed()
    }

    @Test
    fun linhaInteiraEUmaCaixaDeSelecao() = runComposeUiTest {
        setContent { Conteudo(largo = false) }
        onNodeWithText("Estudar Compose").assertIsToggleable().assertIsOff()
        onNodeWithText("Estudar Compose").performClick()
        onNodeWithText("Estudar Compose").assertIsOn()
    }

    @Test
    fun tituloEAnunciadoComoCabecalho() = runComposeUiTest {
        setContent { Conteudo(largo = false) }
        assertEquals(1, onAllNodes(isHeading()).fetchSemanticsNodes().size)
    }
}
```

A estrutura é sempre a mesma:

1. `runComposeUiTest { }` cria o ambiente. É API experimental (`@OptIn(ExperimentalTestApi::class)`).
2. `setContent { }` monta a tela.
3. Encontrar (finder): `onNodeWithText`, `onAllNodesWithText`, `onAllNodes(isHeading())`.
4. Agir: `performClick`, `performTextInput`.
5. Conferir: `assertIsDisplayed`, `assertIsNotEnabled`, `assertIsOn`, `assertIsToggleable`.

O teste procura nós na mesma árvore de semântica que o leitor de tela usa. Por isso ele também testa acessibilidade: com o cartão de 14/09, `assertIsToggleable()` no texto falha com `Failed to assert the following: (ToggleableState is defined)`, porque o texto era um nó sem estado.

Um teste que encontra os elementos pelo texto visível e age como o usuário tende a passar só quando a tela também faz sentido para quem usa leitor de tela.

📖 Ref. Android — Compose testing APIs: https://developer.android.com/develop/ui/compose/testing/apis

5.4 Testar navegação e as duas larguras

Como `Conteudo` recebe `largo`, os testes montam cada layout diretamente:

```
@Test
fun compactaNavegaParaODetalheEVolta() = runComposeUiTest {
    setContent { Conteudo(largo = false) }
    onAllNodesWithText("Detalhes")[0].performClick()
    onNodeWithText("Situação: pendente").assertIsDisplayed()
    onNodeWithText("Marcar como feita").performClick()
    onNodeWithText("Situação: feita").assertIsDisplayed()
    onNodeWithText("Voltar").performClick()
    onNodeWithText("Estudar Compose").assertIsOn()
}

@Test
fun largaMostraListaEDetalheLadoALado() = runComposeUiTest {
    setContent { Conteudo(largo = true) }
    onNodeWithText("Selecione uma tarefa").assertIsDisplayed()
    onAllNodesWithText("Detalhes")[1].performClick()
    onNodeWithText("Situação: pendente").assertIsDisplayed()
    onNodeWithText("Nova tarefa").assertIsDisplayed() // a lista continua na tela
}
```

- O primeiro percorre navegação, argumento, mudança de estado e pilha de retorno num teste só: abre o detalhe, marca, volta e confere a lista.
- O segundo confere o modo largo sem precisar de uma janela larga.
- No alvo desktop, também dá para testar `App()` com um tamanho de janela definido, usando `runDesktopComposeUiTest(width = 412, height = 915)` num teste de `desktopTest`. Foi assim que os limites de 599 e 600 da seção 2.3 foram medidos.

> Erro comum: `onNodeWithText` quando há mais de um nó com o texto. Com dois botões "Detalhes", o teste falha com `Expected exactly '1' node but found '2' nodes that satisfy: (Text + InputText + EditableText contains 'Detalhes' ...)`. Use `onAllNodesWithText("Detalhes")[0]`, ou um critério mais específico.

> Erro comum: importar `onAllNodes`. Ele é um método do receptor de `runComposeUiTest`, não uma função solta; o import `androidx.compose.ui.test.onAllNodes` dá `Unresolved reference 'onAllNodes'`.

> Erro comum: montar `App()` no teste e depender do tamanho da janela de teste. O resultado muda com o ambiente. Separe a decisão (`App`) do conteúdo (`Conteudo(largo)`) e teste os dois modos.

5.5 No MUSI: as mesmas ideias, noutro domínio

O projeto de referência (`github.com/fmarquesfilho/musi`, pasta `app/`) aplica tudo isto a um acervo musical, com o alvo Android ligado nesta sprint:

Para ver	Onde, no MUSI
Rotas tipadas com argumento de texto	`App.kt`: `Acervo` e `DetalheDaObra(val id: String)`
Deep link	`musi://obra/{id}`, no grafo e no `AndroidManifest.xml`
Duas larguras	estreita navega; larga mostra acervo e obra lado a lado, sem navegar
Tema claro e escuro	`TemaMusi`, com `isSystemInDarkTheme()`
Formulário validado	a tela chama as regras do módulo `shared` — as mesmas que a API usa antes de gravar
Testes de interface	8, em `commonTest`, rodando no alvo desktop, no CI

O id da rota é `String`, e não `Int` como no exemplo, porque no domínio do MUSI a obra é identificada por texto. Rota tipada não obriga a usar inteiro: o argumento é uma propriedade, com o tipo do domínio.

Dois cuidados que apareceram ao ligar o alvo Android, e que valem para o projeto de vocês:

> Erro comum: `./gradlew build` quebra os testes de interface. O `build` de um módulo com alvo Android roda também a variante de teste **unitário** do Android (`testDebugUnitTest`), que executa os testes de `commonTest` numa JVM sem Android de verdade: os testes de interface falham com `NullPointerException ... android.os.Build.FINGERPRINT is null`. Eles pertencem ao alvo desktop (ou a `androidInstrumentedTest`, com aparelho).

> Erro comum: AGP 9 recusa o projeto. `com.android.application` não combina mais com o plugin Kotlin Multiplatform ("not compatible ... since AGP 9.0") sem `android.builtInKotlin=false` e `android.newDsl=false` no `gradle.properties` — que é o que o exemplo desta disciplina já traz. Para um módulo de **biblioteca** compartilhada, o caminho novo é o plugin `com.android.kotlin.multiplatform.library`.

---

6. Oficina: testes para o projeto

A rubrica pede pelo menos 5 testes de interface cobrindo as telas principais e a validação do formulário. Um roteiro para o projeto do grupo:

1. Separe as regras das telas em funções puras e cubra cada uma com `kotlin.test`: são os testes mais baratos.
2. Para cada tela principal, um teste de interface que monta a tela com dados fixos e confere o que aparece.
3. Para cada formulário, um teste com entrada inválida (botão desabilitado ou mensagem de erro) e um com entrada válida (o item aparece).
4. Um teste de navegação por fluxo importante: abre, age, volta, confere.
5. Um teste de acessibilidade por componente próprio interativo: `assertIsToggleable`, `assertHasClickAction` ou a presença de `contentDescription` (`onNodeWithContentDescription`).

Para que as telas sejam testáveis, elas precisam receber estado e devolver eventos, sem criar o próprio estado nem conhecer o `navController`. É o mesmo estado elevado de 14/09; o teste só torna o benefício visível.

A rubrica pede os testes verdes no CI. O workflow da Sprint 0 (tarefa T7, com `ktlint` e `detekt`) ganha mais um passo: `./gradlew :composeApp:desktopTest`. Os testes de interface não abrem janela nem precisam de emulador, mas o Skia, que desenha o Compose no desktop, depende de bibliotecas gráficas do sistema. Num container Linux mínimo, sem tela, os 5 testes de interface falharam com `UnsatisfiedLinkError: ... libGL.so.1: cannot open shared object file` (e depois `libEGL.so.1`); com `libgl1`, `libegl1` e `libfontconfig1` instaladas, os 8 passaram. No workflow:

```
- run: sudo apt-get update && sudo apt-get install -y libgl1 libegl1 libfontconfig1
- run: ./gradlew :composeApp:desktopTest
```

Isso basta para os testes, que rodam no alvo desktop. Quem quiser **também** gerar o APK no CI precisa do SDK do Android no runner, e aí vale um cuidado: o identificador da plataforma tem a versão completa.

No MUSI, `platforms;android-37` falhou com `Failed to find package`, e o certo é `platforms;android-37.0` — o mesmo que está no `package.xml` da plataforma instalada.

```
- uses: android-actions/setup-android@v3
  with:
    packages: 'platforms;android-37.0 build-tools;36.0.0'
- run: ./gradlew :composeApp:assembleDebug
```

---

7. Exercícios e dúvidas frequentes

7.1 Perguntas de fixação

1. Qual a diferença entre layout responsivo e adaptativo?
2. Por que a decisão de layout usa a largura da janela e não a orientação?
3. O que `MergeDescendants` faz na árvore de semântica, e por que o cartão precisa disso?
4. Por que `onCheckedChange = null` na `Checkbox` dentro de uma linha `toggleable`?
5. Por que a rota `Detalhe` carrega só o id, e não a tarefa inteira?
6. O que acontece com o Voltar do sistema depois de abrir o app por deep link?
7. Por que `Conteudo` recebe `largo` como parâmetro em vez de calcular sozinho?
8. O que um teste de interface tem a ver com o leitor de tela?

7.2 Exercícios práticos

1. Adicione um botão "Apagar" com ícone no detalhe, com `contentDescription`, e um teste que o encontre com `onNodeWithContentDescription`.
2. Limite a largura do painel de detalhe a 600 dp no modo largo e confira no desktop com a janela maximizada.
3. Crie uma terceira classe de layout: na largura expandida (840 dp ou mais), mostre também um resumo (quantas feitas, quantas pendentes). Teste as três larguras.
4. Adicione um deep link `tarefas://nova?titulo=...` que abra a lista com o campo preenchido. Pense em qual rota e qual argumento usar.
5. Ative o TalkBack no emulador e percorra as duas telas. Anote o que é anunciado e corrija o que não fizer sentido.
6. Escreva um teste que falhe se alguém trocar o `toggleable` da linha por um `clickable` só no texto.

7.3 Dúvidas frequentes

- O Android Studio do laboratório não abre o projeto. Como faço? O Ladybug (2024) aceita AGP até 8.7, e o exemplo usa 9.1. Use o Codespaces com o Android Studio na área de trabalho do

navegador, ou gere o APK no Codespaces e instale no emulador do laboratório (ver *Como abrir e rodar* em `exemplos/tarefas-compose/PASSOS.md`).

- O Android Studio ou o emulador ficam pesados no meu computador. O que usar? Para construir as telas e rodar os testes, o alvo desktop com Hot Reload basta, sem Android Studio nem emulador. Para o `@Preview` e o APK, use o Codespaces. Para ver o app no Android, instale o APK num celular: baixe o arquivo do Codespace e abra no aparelho, ou use `adb install`.
- Como é a primeira vez no Codespaces? A criação leva alguns minutos. A área de trabalho abre na aba Portas, porta 6080 (senha `vscode`). `mise run studio` baixa e abre o Android Studio; ele pergunta sobre o envio de estatísticas e se você confia no projeto (Trust Project), e depois sincroniza o Gradle. O primeiro `@Preview` pede Build & Refresh, que leva alguns minutos. `mise run app:celular` abre o app numa janela de celular, com Hot Reload. O uso gratuito mensal é limitado: pare o Codespace ao terminar.
- Posso usar o Codespaces no projeto do grupo? Sim. Copie a pasta `.devcontainer/` para o repositório do grupo; ela traz o JDK, o SDK do Android e a área de trabalho no navegador.
- O deep link funciona no desktop? Não da mesma forma: não há `intent-filter` nem `adb` no desktop. O `navDeepLink` do grafo existe em `commonMain`, mas abrir um endereço a partir de fora do app depende do sistema de cada plataforma.
- Navigation 3 não é o mais novo? Existe uma biblioteca nova (Navigation 3), com a pilha como uma lista que o próprio app controla. O curso usa o Navigation Compose (2.x), que é o que a rubrica pede e tem rotas tipadas e deep links.
- `remember` ou `rememberSaveable` para a lista? No exemplo, a lista usa `remember`, e o texto e a seleção usam `rememberSaveable`. Guardar listas de objetos com `rememberSaveable` exige um `Saver`; o caminho do curso é levar esse estado para um `ViewModel` na Sprint 2.
- Os testes de interface precisam de emulador? Não: rodam no alvo desktop. Testes instrumentados no emulador existem, mas são mais lentos e não são pedidos na Sprint 1.
- Onde vejo a árvore de semântica? Num teste, `onRoot().printToString()` devolve o texto mostrado na seção 3.2. No Android Studio, o Layout Inspector também mostra as propriedades de semântica de cada nó.

Referências

Responsividade e adaptatividade

- Android — Use window size classes: https://developer.android.com/develop/ui/compose/layouts/adaptive/use-window-size-classes
- Android — Adaptive layouts: https://developer.android.com/develop/ui/compose/layouts/adaptive
- Compose Multiplatform — Adaptive layouts: https://kotlinlang.org/docs/multiplatform/compose-adaptive-layouts.html
- Android — Window insets in Compose: https://developer.android.com/develop/ui/compose/system/insets

Acessibilidade

- Android — Accessibility in Compose: https://developer.android.com/develop/ui/compose/accessibility
- Android — Semantics: https://developer.android.com/develop/ui/compose/accessibility/semantics
- Android — Merging and clearing semantics: https://developer.android.com/develop/ui/compose/accessibility/merging-clearing
- Android — Key steps to improve Compose accessibility: https://developer.android.com/develop/ui/compose/accessibility/key-steps
- Android — Testing accessibility in Compose: https://developer.android.com/develop/ui/compose/accessibility/testing
- Compose Multiplatform — Accessibility: https://kotlinlang.org/docs/multiplatform/compose-accessibility.html
- WCAG 2.2 — Target Size (Minimum): https://www.w3.org/WAI/WCAG22/Understanding/target-size-minimum.html

Navegação

- Compose Multiplatform — Navigation: https://kotlinlang.org/docs/multiplatform/compose-navigation.html
- Compose Multiplatform — Navigation and routing: https://kotlinlang.org/docs/multiplatform/compose-navigation-routing.html
- Compose Multiplatform — Deep links: https://kotlinlang.org/docs/multiplatform/compose-navigation-deep-links.html
- Android — Type safety in Navigation Compose: https://developer.android.com/guide/navigation/design/type-safety
- Android — Back stack: https://developer.android.com/guide/navigation/backstack
- Android — Create a deep link for a destination: https://developer.android.com/guide/navigation/navigation-deep-link

Testes

- Compose Multiplatform — Testing Compose UI: https://kotlinlang.org/docs/multiplatform/compose-test.html
- Kotlin Multiplatform — Run tests: https://kotlinlang.org/docs/multiplatform/multiplatform-run-tests.html
- kotlin.test: https://kotlinlang.org/api/core/kotlin-test/
- Android — Testing your Compose layout: https://developer.android.com/develop/ui/compose/testing
- Android — Compose testing APIs: https://developer.android.com/develop/ui/compose/testing/apis
- Android — Common testing patterns: https://developer.android.com/develop/ui/compose/testing/common-patterns